

Expense Tracker — Phase 1

Backend API Test Report (Postman)

Dalil Training Academy — Full Stack Web Development

1. Overview

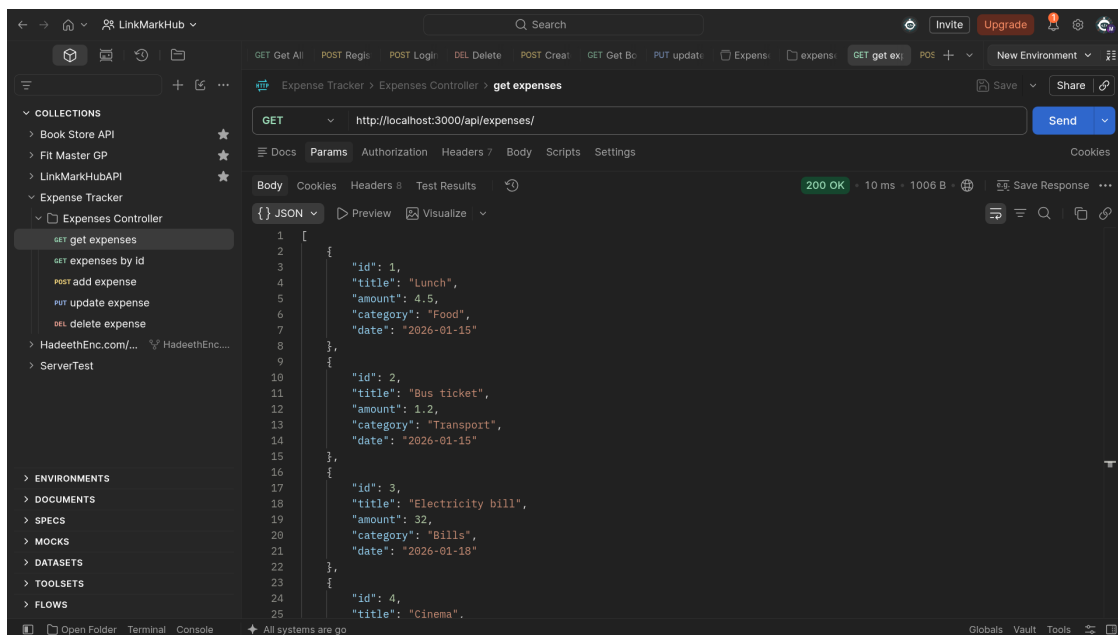
This report documents the testing of the 5 backend endpoints built in Phase 1 of the Expense Tracker project (Node.js + Express + PostgreSQL). Every endpoint was tested with Postman for both its success case and its expected error cases (invalid input, invalid id, and not-found id).

Method	Path	Function	Success	Errors
GET	/api/expenses	Return all expenses	200	—
GET	/api/expenses/:id	Return one expense	200	400, 404
POST	/api/expenses	Add a new expense	201	400
PUT	/api/expenses/:id	Update an expense	200	400, 404
DELETE	/api/expenses/:id	Delete an expense	200	400, 404

2. Sample Requests & Responses

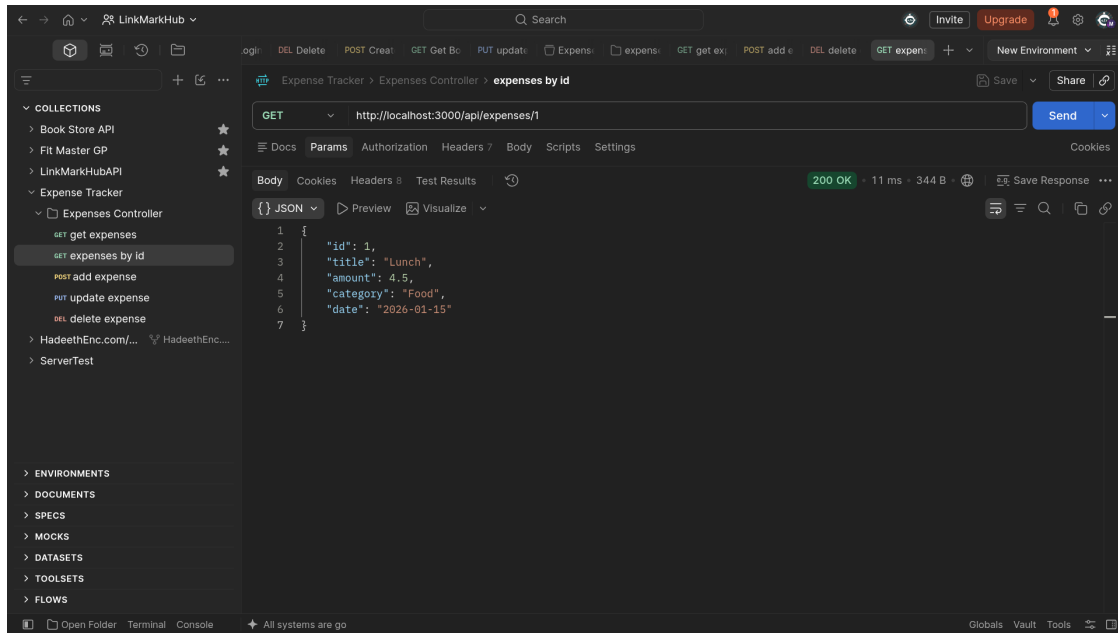
2.1 GET /api/expenses — Get All Expenses

Returns every expense stored in the database, with amount as a proper number and date in YYYY-MM-DD format. Status: 200 OK.



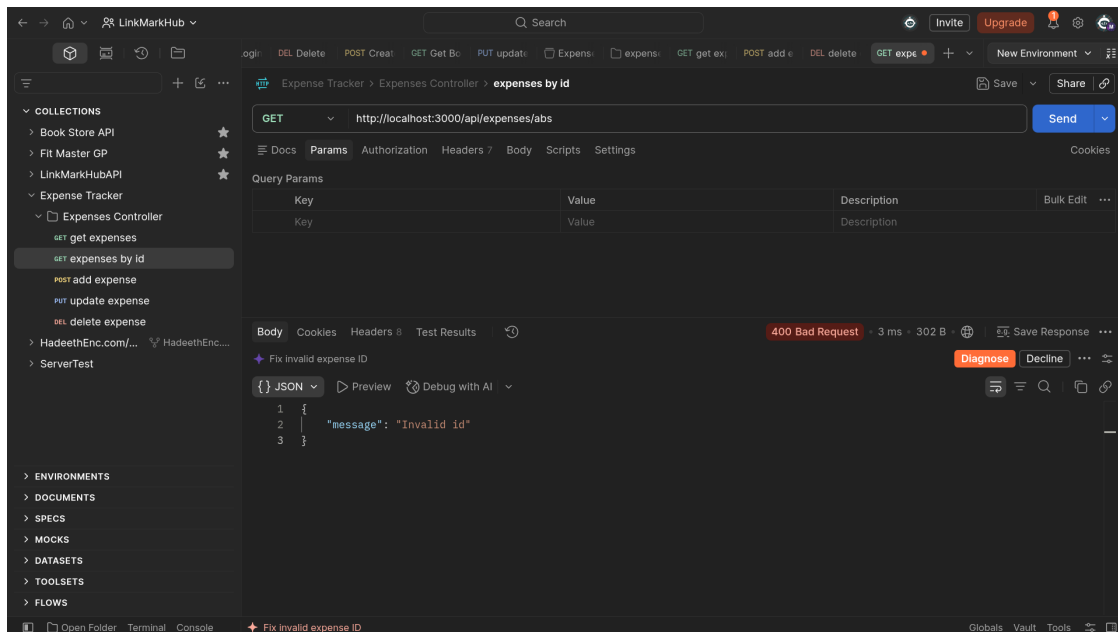
2.2 GET /api/expenses/:id — Get Expense by ID

Returns a single expense matching the given id. Status: 200 OK.



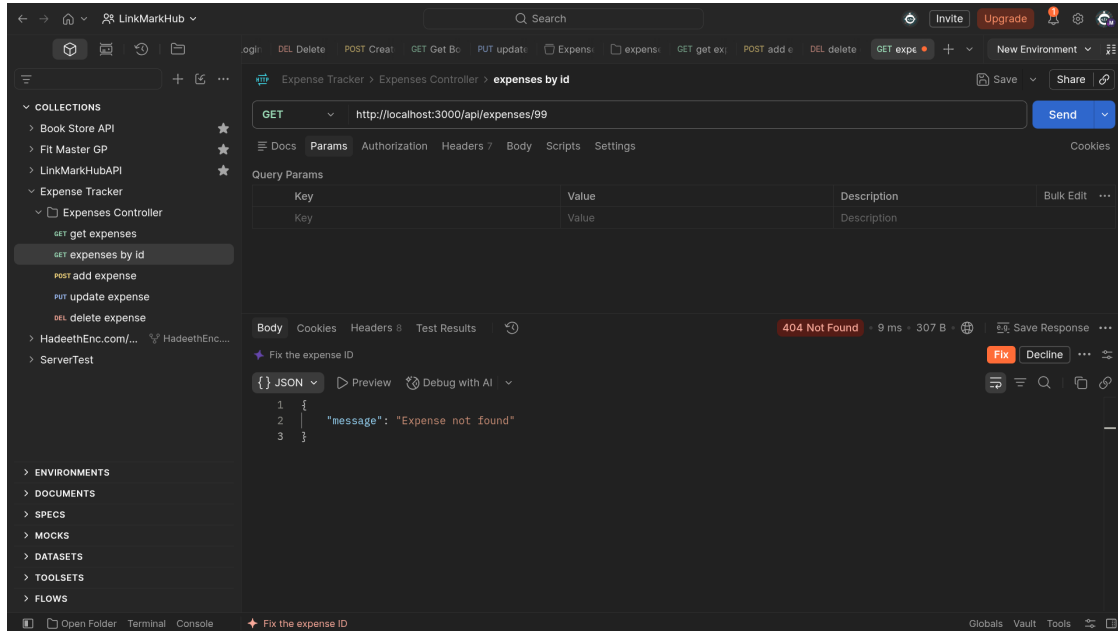
2.3 GET /api/expenses/:id — Invalid ID (400)

Requesting a non-numeric id (e.g. /api/expenses/abs) is rejected before it ever reaches the database, returning a clear 400 error.



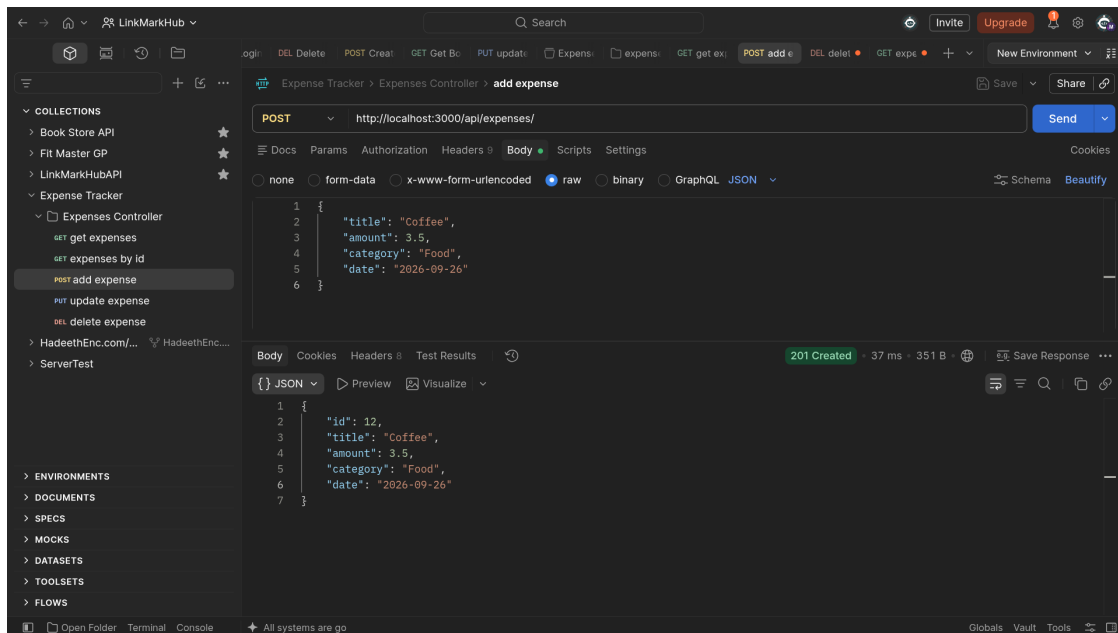
2.4 GET /api/expenses/:id — Not Found (404)

Requesting an id that is valid but doesn't exist in the table (e.g. /api/expenses/99) returns 404 with a clear message.



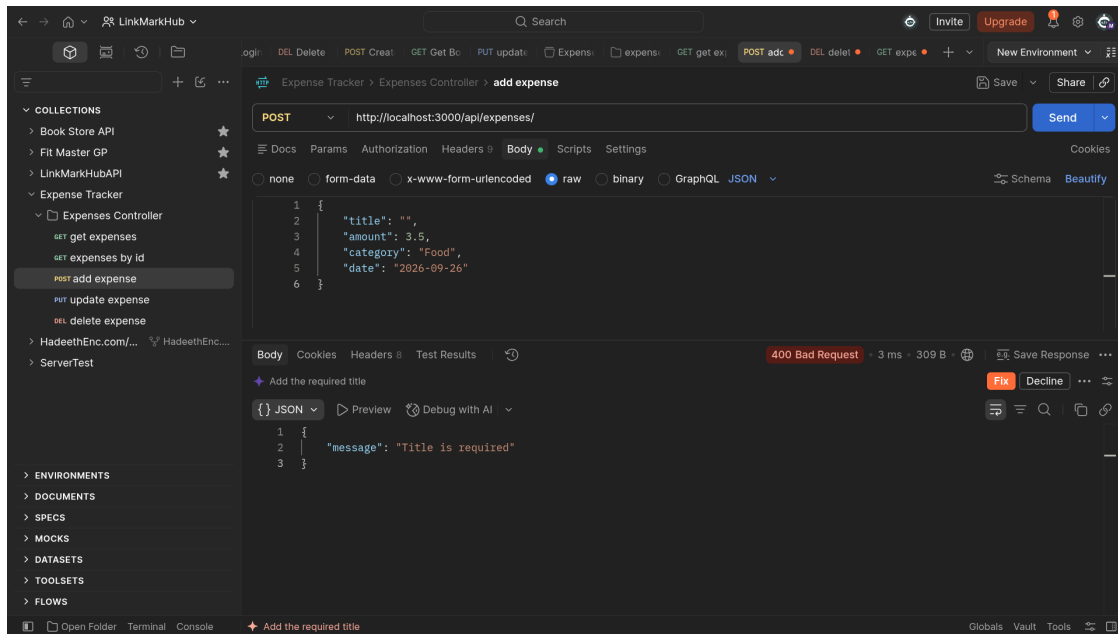
2.5 POST /api/expenses — Add Expense (Success)

Sending a valid expense object returns 201 Created along with the new row, including the id generated by the database.



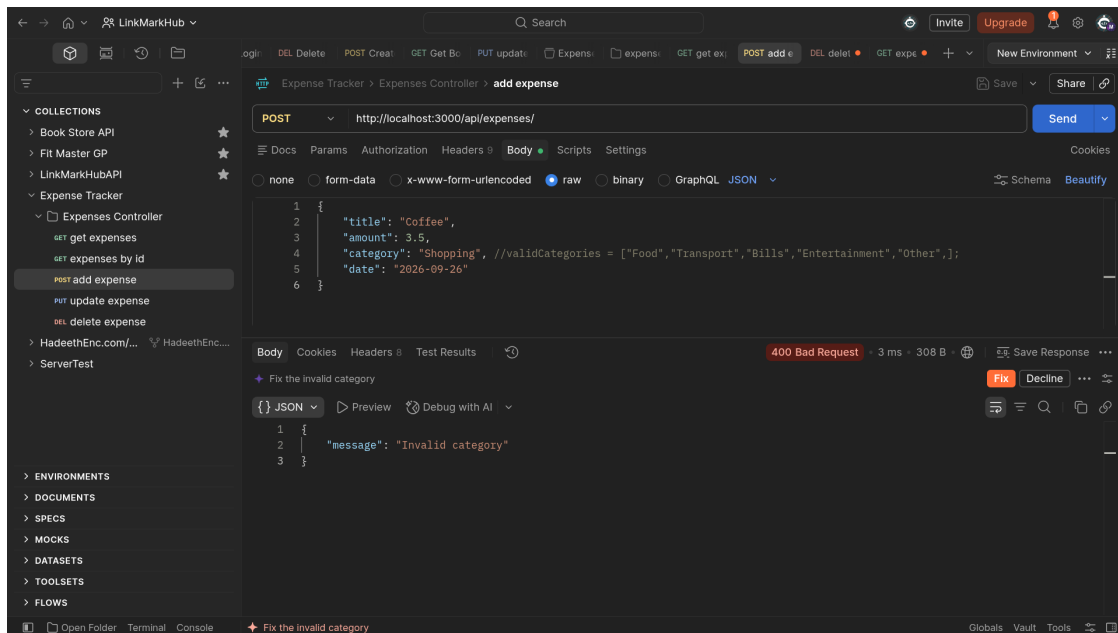
2.6 POST /api/expenses — Empty Title (400)

Submitting an empty title fails validation and returns 400 with a message explaining the problem, before any database write happens.



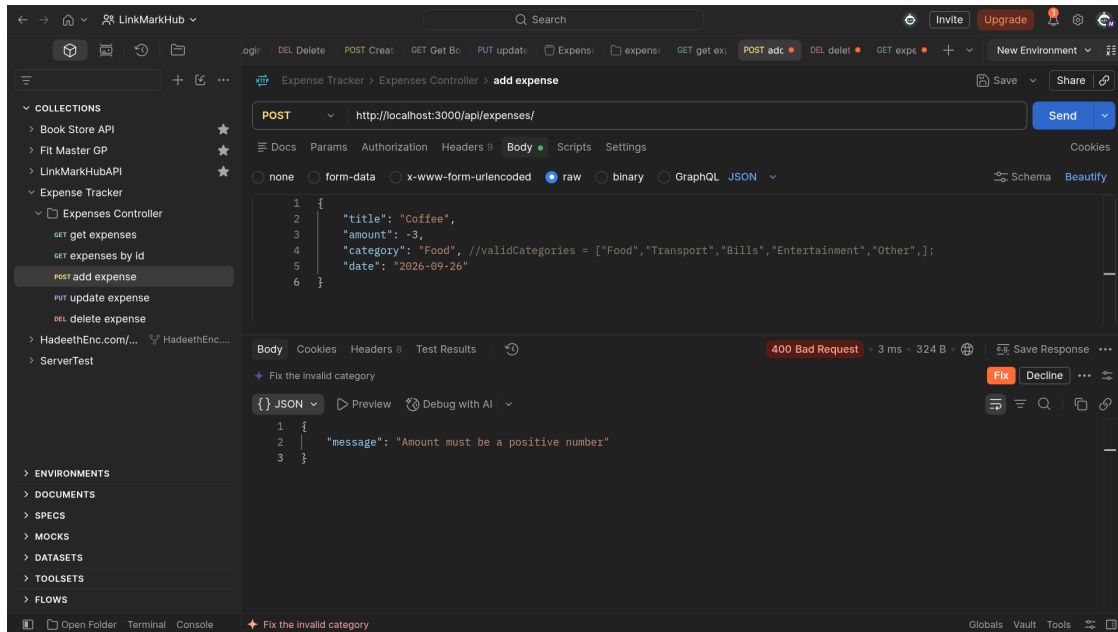
2.7 POST /api/expenses — Invalid Category (400)

A category outside the five allowed values (Food, Transport, Bills, Entertainment, Other) is rejected with 400.



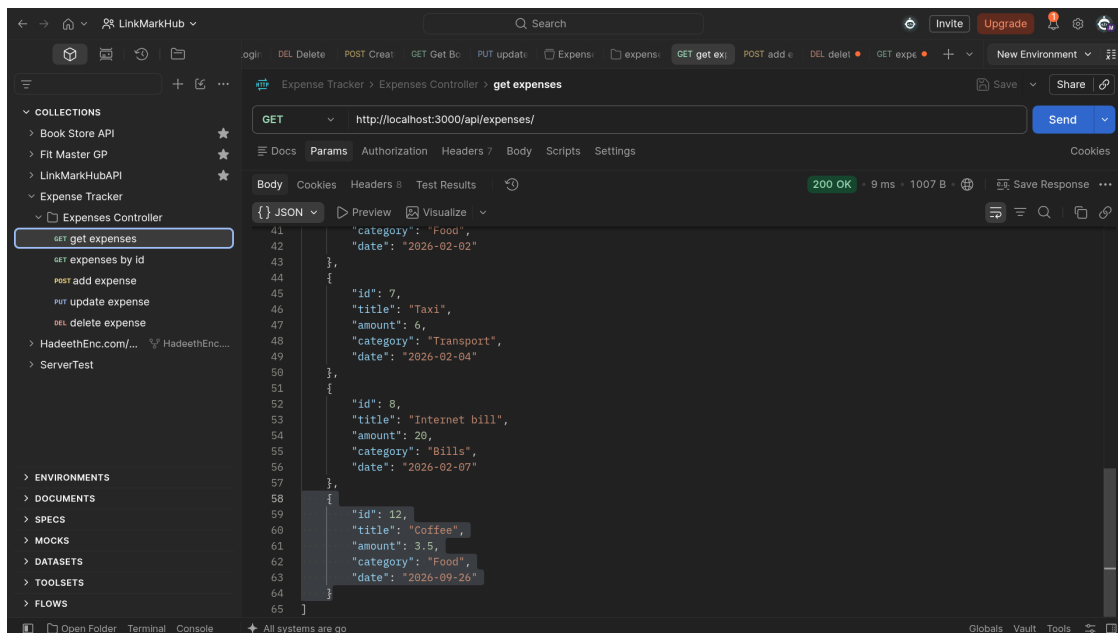
2.8 POST /api/expenses — Negative Amount (400)

An amount of zero or less fails validation and returns 400.



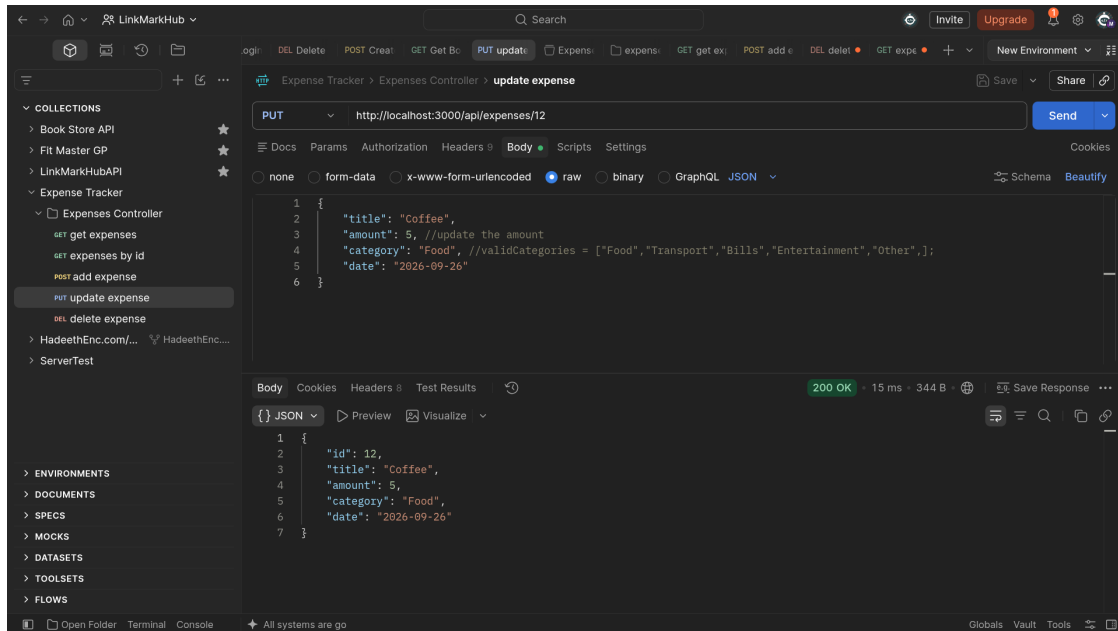
2.9 Verifying the Insert

Calling GET /api/expenses again after the successful POST confirms the new expense (id 12, "Coffee") is now persisted in the database.



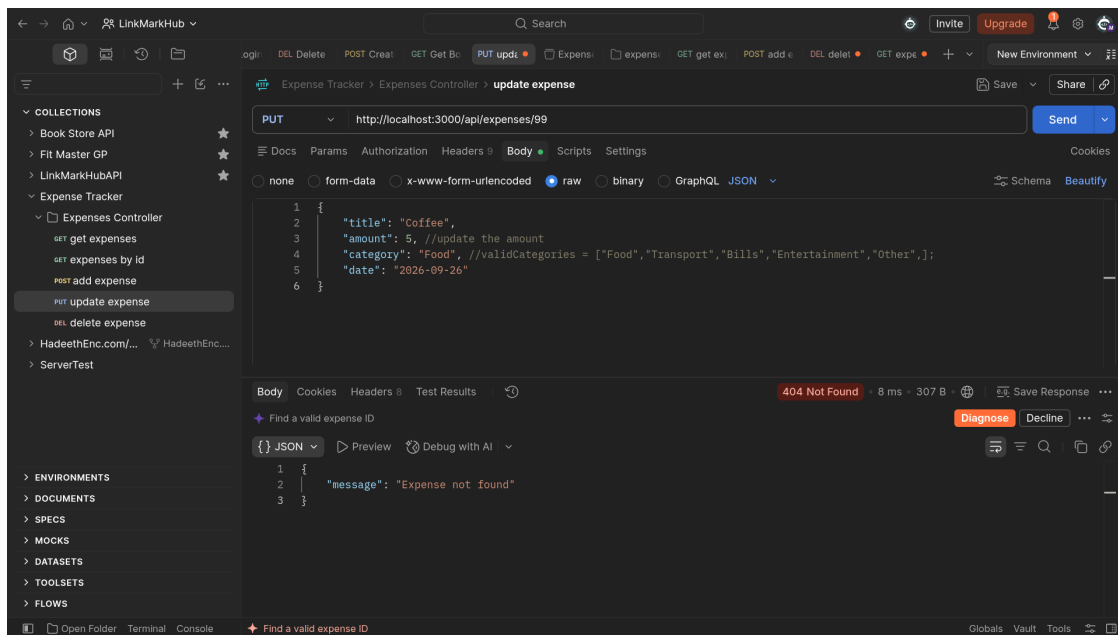
2.10 PUT /api/expenses/:id — Update Expense (Success)

Updating the amount of an existing expense returns 200 OK with the updated row.



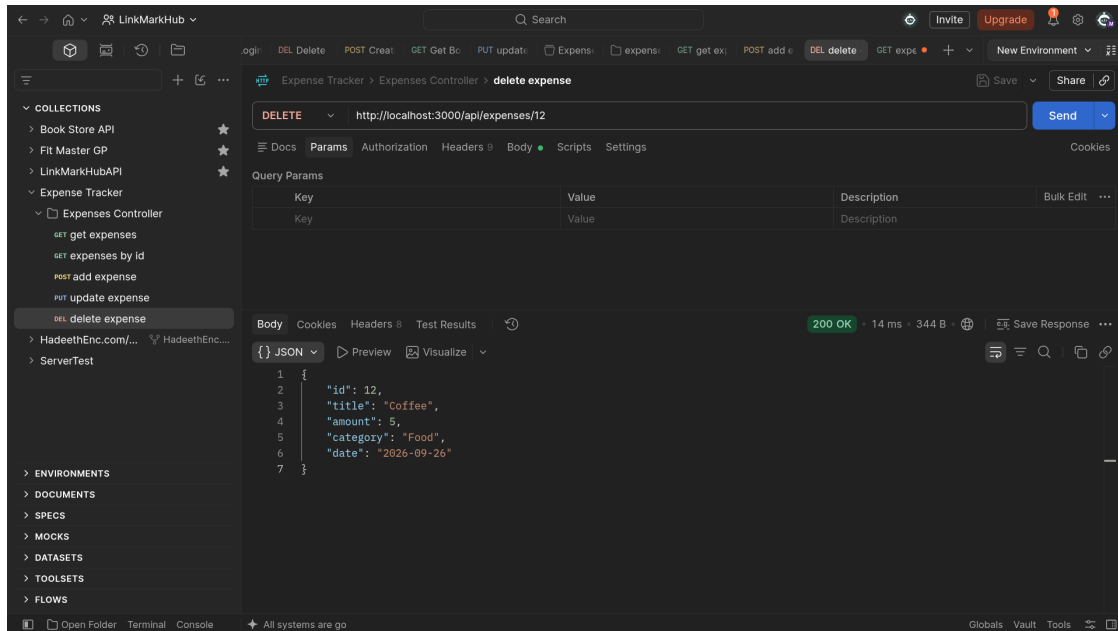
2.11 PUT /api/expenses/:id — Not Found (404)

Trying to update an id that doesn't exist returns 404, instead of silently doing nothing or crashing the server.



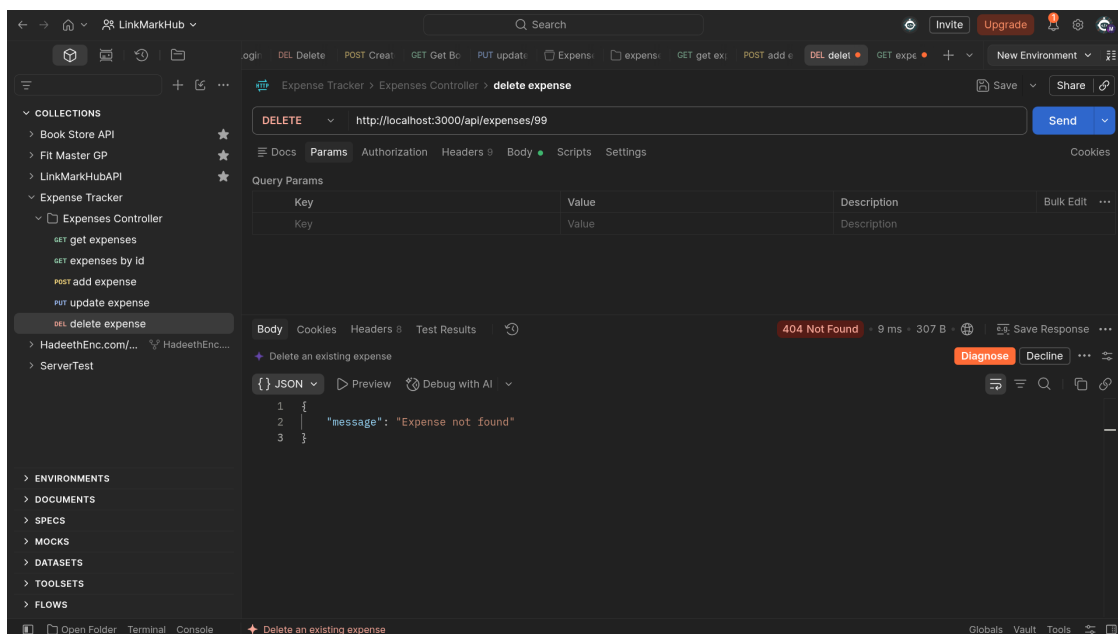
2.12 DELETE /api/expenses/:id — Delete Expense (Success)

Deleting an existing expense returns 200 OK along with the row that was removed.



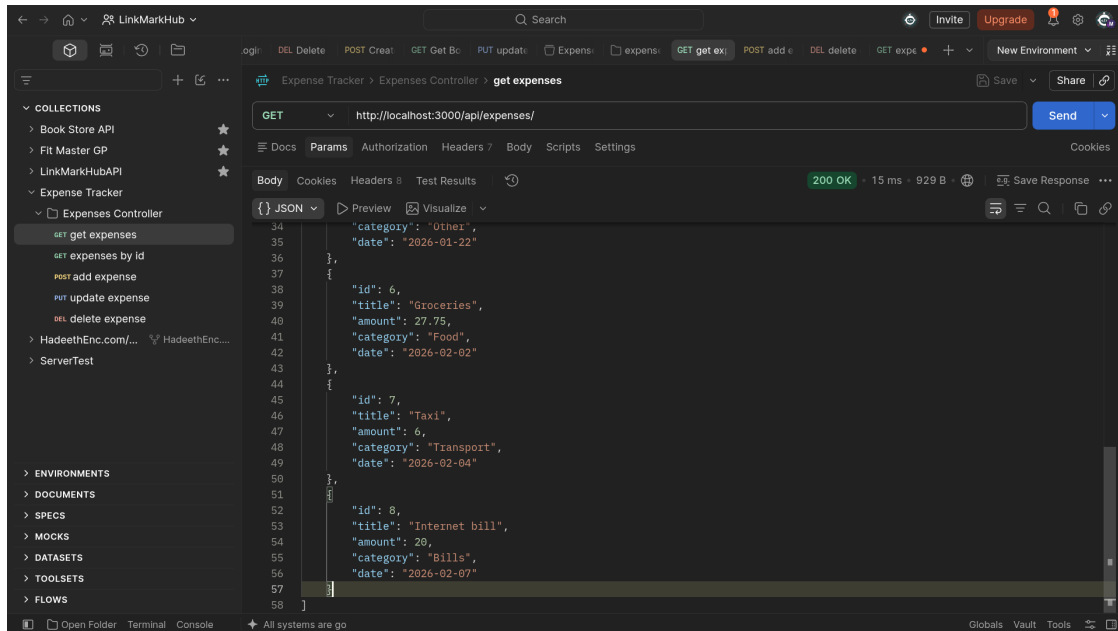
2.13 DELETE /api/expenses/:id — Not Found (404)

Deleting the same id a second time (already removed) correctly returns 404 instead of a false success.



2.14 Verifying the Deletion

Calling GET /api/expenses again confirms the deleted expense no longer appears in the list.



3. HTTP Status Codes Used

The same status codes were used consistently across all 5 endpoints, and every error response returns a JSON body with a clear message field — never a raw HTML error page.

Code	Meaning	When it happens
200 OK	Success	GET, PUT, and DELETE operations that succeed
201 Created	Resource created	POST /api/expenses with valid data
400 Bad Request	Invalid input	Missing/invalid title, amount, category, date, or a non-numeric id
404 Not Found	Resource missing	GET, PUT, or DELETE with an id that doesn't exist
500 Internal Server Error	Server error	Unexpected failure caught by try/catch

4. Conclusion

All 5 endpoints (GET all, GET by id, POST, PUT, DELETE) were tested for both their success path and their main failure paths (invalid id, missing/invalid body fields, and not-found id). Every query uses parameterized values (\$1, \$2, ...), amount and date are returned in clean, usable formats, and no request produces an unhandled server crash.